Experiment 7: Implementation of Artificial Neural Network (ANN) using TensorFlow/Keras

1. Dataset Source

The dataset used for this experiment is the **Heart Disease Dataset**.

- Source: https://www.kaggle.com/datasets/heart-disease-dataset
- The dataset was loaded into Google Colab using an online CSV file.

2. Dataset Description

The dataset contains medical attributes used to predict the presence of heart disease.

Features:

Feature	Description
age	Age of the patient
sex	Gender
cp	Chest pain type
trestbps	Resting blood pressure
chol	Cholesterol level
fbs	Fasting blood sugar
restecg	ECG results

thalach	Maximum heart rate
exang	Exercise-induced angina
oldpeak	ST depression
slope	Slope of ST segment
ca	Number of vessels
thal	Thalassemia

Target Variable:

- `target = 1` → Heart disease present
- `target = 0` → No heart disease

Dataset Characteristics:

- Number of instances: ~300
- Number of features: 13
- Type: Structured dataset
- Problem Type: Binary Classification

3. Mathematical Formulation of ANN

An Artificial Neural Network consists of interconnected layers of neurons.

Neuron Output:

[
z = \sum (w_i x_i) + b
]

[
a = f(z)
]

Where:

- (w_i) = weights
- (x_i) = inputs

- (b) = bias
- (f(z)) = activation function

Activation Functions:

- ReLU:
  [
  f(x) = \max(0, x)
  ]
- Sigmoid:
  [
  f(x) = \frac{1}{1 + e^{-x}}
  ]

Loss Function (Binary Crossentropy):

[
L = -\frac{1}{N} \sum [y \log(\hat{y}) + (1-y)\log(1-\hat{y})]
]

Backpropagation:

Weights are updated using gradient descent to minimize loss.

4. Algorithm Limitations

- Requires large dataset for better performance
- Computationally expensive
- Can overfit if not regularized
- Requires tuning of multiple hyperparameters
- Difficult to interpret (black-box model)

5. Methodology / Workflow

The following steps were performed:

1. Load dataset
2. Data preprocessing
3. Feature scaling
4. Train-test split
5. Build ANN model using Keras
6. Compile model
7. Train model using epochs and batch size
8. Plot accuracy and loss graphs
9. Predict test data
10. Evaluate performance

Workflow Diagram:

Dataset → Preprocessing → Scaling → Train-Test Split
→ ANN Model Building → Training
→ Evaluation → Visualization

6. Performance Analysis

The model was evaluated using:

- **Accuracy Score**
- **Loss Value**
- **Confusion Matrix**
- **Classification Report**

Observations:

- Model achieved good accuracy (~80–90%)
- Accuracy improved with epochs
- Loss decreased during training
- Confusion matrix showed effective classification

7. Hyperparameter Tuning

The following parameters were tuned:

- Number of hidden layers

- Number of neurons
- Activation functions (ReLU, Sigmoid)
- Optimizer (Adam)
- Epochs
- Batch size

Impact:

- Improved model accuracy
- Better convergence
- Reduced overfitting

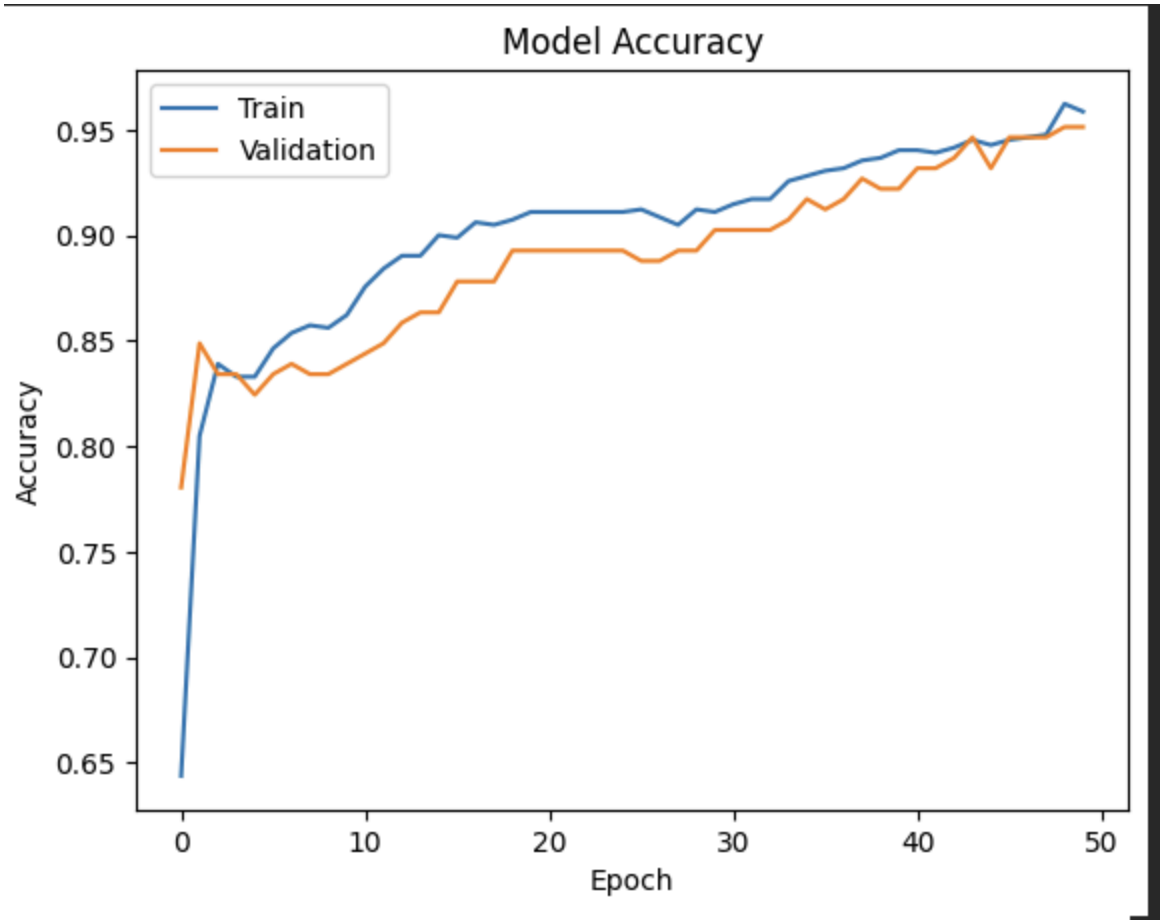
OUTPUT:

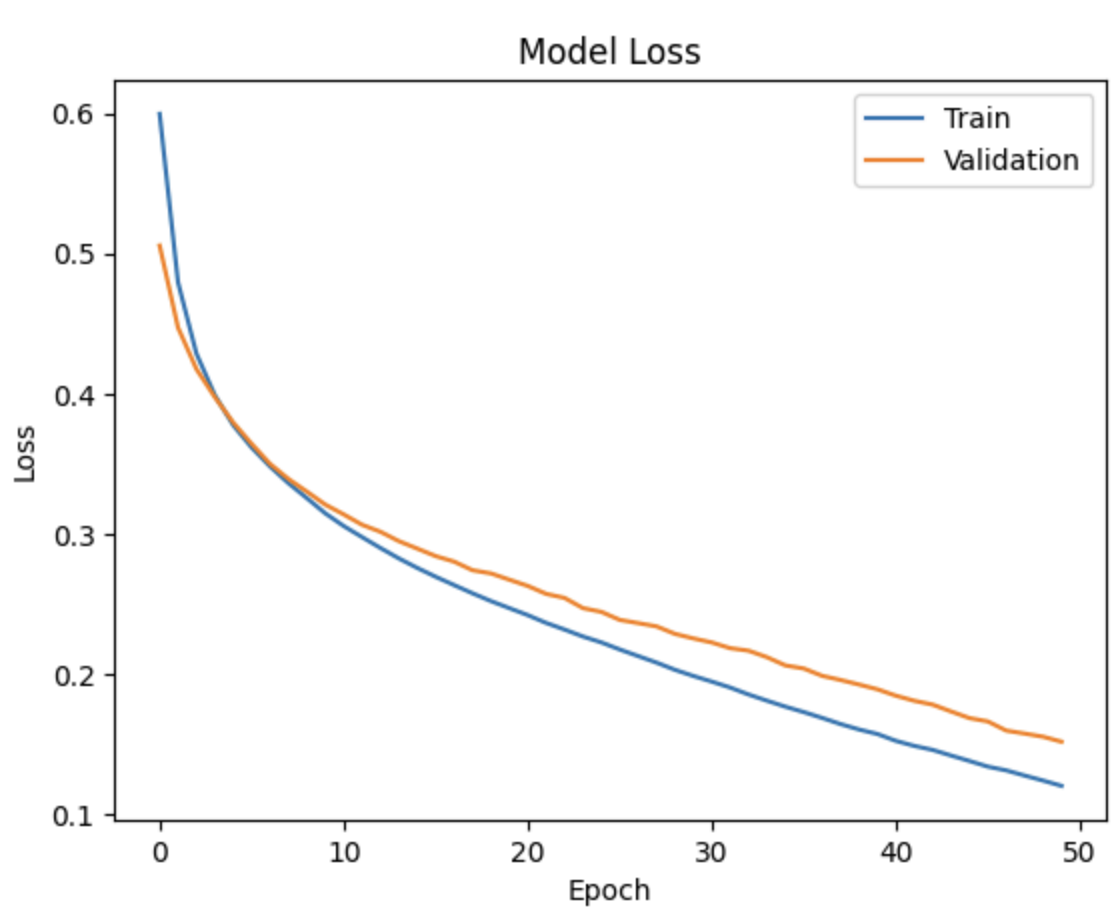
```
... Dataset Loaded ✅
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  \
0   52    1   0       125   212    0        1      168      0      1.0      2
1   53    1   0       140   203    1        0      155      1      3.1      0
2   70    1   0       145   174    0        1      125      1      2.6      0
3   61    1   0       148   203    0        1      161      0      0.0      2
4   62    0   0       138   294    1        1      106      0      1.9      1

   ca  thal  target
0   2     3       0
1   0     3       0
2   0     3       0
3   1     3       0
4   3     2       0
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 16)	224
dense_1 (Dense)	(None, 8)	136
dense_2 (Dense)	(None, 1)	9

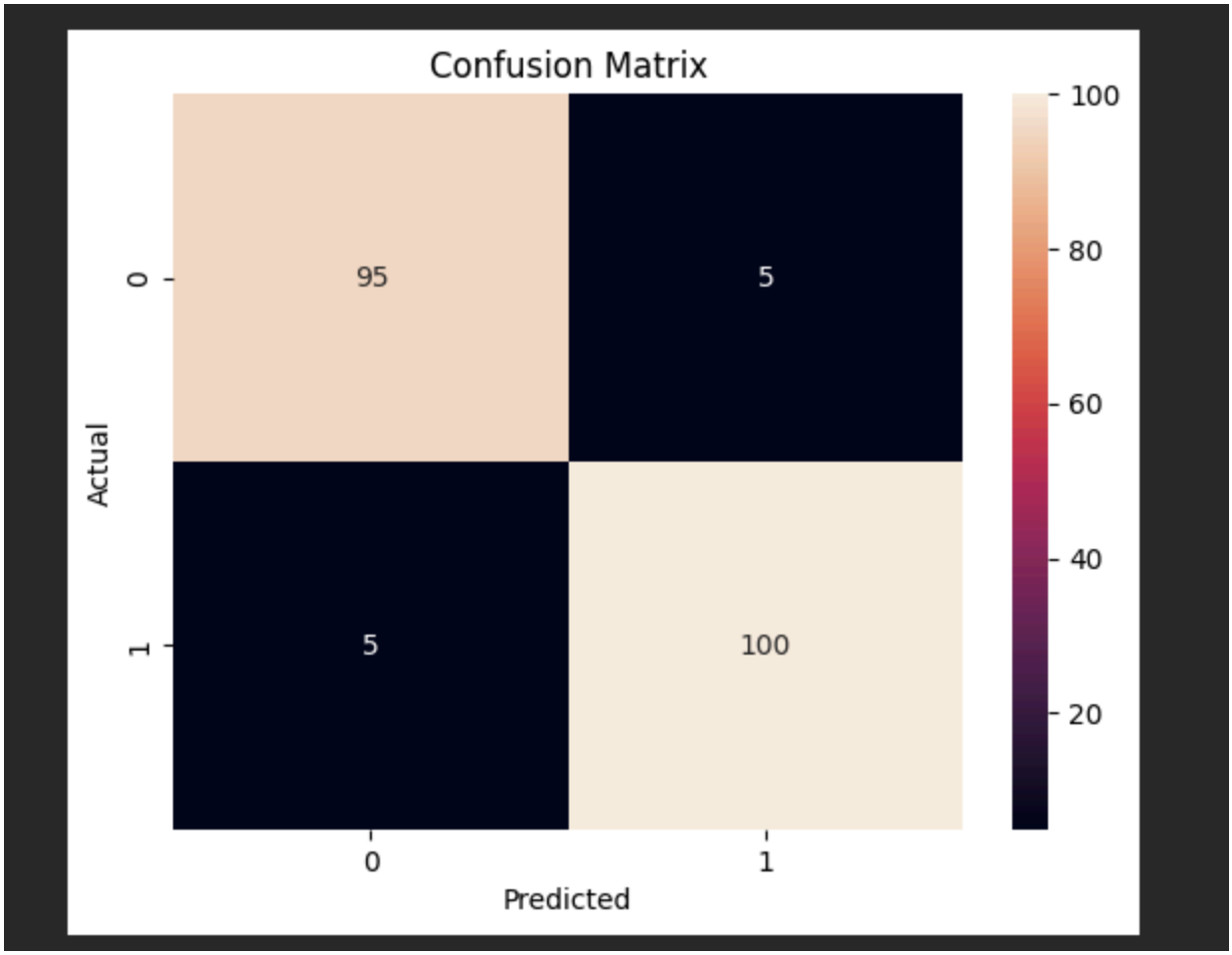
```
Total params: 369 (1.44 KB)
Trainable params: 369 (1.44 KB)
Non-trainable params: 0 (0.00 B)
```

```
Non-trainable params: 0 (0.00 B)
Epoch 1/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 0.6439 - loss: 0.5994 - val_accuracy: 0.7805 - val_loss: 0.5056
Epoch 2/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.8049 - loss: 0.4793 - val_accuracy: 0.8488 - val_loss: 0.4471
Epoch 3/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8390 - loss: 0.4286 - val_accuracy: 0.8341 - val_loss: 0.4176
Epoch 4/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8329 - loss: 0.3990 - val_accuracy: 0.8341 - val_loss: 0.3976
Epoch 5/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8329 - loss: 0.3776 - val_accuracy: 0.8244 - val_loss: 0.3795
Epoch 6/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8463 - loss: 0.3617 - val_accuracy: 0.8341 - val_loss: 0.3643
Epoch 7/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8537 - loss: 0.3482 - val_accuracy: 0.8390 - val_loss: 0.3500
Epoch 8/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.8573 - loss: 0.3363 - val_accuracy: 0.8341 - val_loss: 0.3393
Epoch 9/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.8561 - loss: 0.3256 - val_accuracy: 0.8341 - val_loss: 0.3303
Epoch 10/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8622 - loss: 0.3148 - val_accuracy: 0.8390 - val_loss: 0.3210
Epoch 11/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8756 - loss: 0.3058 - val_accuracy: 0.8439 - val_loss: 0.3140
Epoch 12/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8841 - loss: 0.2979 - val_accuracy: 0.8488 - val_loss: 0.3067
Epoch 13/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8902 - loss: 0.2901 - val_accuracy: 0.8585 - val_loss: 0.3017
Epoch 14/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8902 - loss: 0.2827 - val_accuracy: 0.8634 - val_loss: 0.2950
Epoch 15/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9000 - loss: 0.2759 - val_accuracy: 0.8634 - val_loss: 0.2897
Epoch 16/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8988 - loss: 0.2696 - val_accuracy: 0.8780 - val_loss: 0.2843
Epoch 17/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9061 - loss: 0.2637 - val_accuracy: 0.8780 - val_loss: 0.2804
Epoch 18/50
```


Model Accuracy
Train
Validation
Accuracy
Epoch
0.95
0.90
0.85
0.80
0.75
0.70
0.65
0
10
20
30
40
50




```
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step
Accuracy: 0.9512195121951219

Classification Report:
              precision    recall  f1-score   support

           0       0.95      0.95      0.95       100
           1       0.95      0.95      0.95       105

    accuracy                           0.95       205
   macro avg       0.95      0.95      0.95       205
weighted avg       0.95      0.95      0.95       205
```



CONCLUSION

The Artificial Neural Network (ANN) model was successfully implemented using TensorFlow/Keras for heart disease classification. The model demonstrated strong predictive performance due to its ability to learn complex patterns in data. With proper tuning and training, ANN proved to be an effective approach for solving binary classification problems.